

# MARK HANEY

## STAFF SOFTWARE ENGINEER | .NET REACT FULL-STACK ENGINEER | CLOUD-NATIVE MICROSERVICES LEAD

Dardanelle, AR

<https://www.linkedin.com/in/markhaney-33b675ba>

+1 (430) 279-2231 | [markhaney.dev@outlook.com](mailto:markhaney.dev@outlook.com)

### PROFESSIONAL SUMMARY

Staff Software Engineer and Full-Stack Technical Lead with 12+ years of experience building SaaS, property technology, civic technology, nonprofit CRM, and managed-services platforms using **C#**, **ASP.NET Core 6–8**, **React 16–18**, **TypeScript 4–5**, SQL Server, PostgreSQL, MongoDB, Azure, AWS, Docker, Kubernetes, and CI/CD pipelines. Strong background in modernizing legacy .NET systems, designing microservices, improving API performance, reducing production defects, mentoring engineers, and delivering secure, scalable, cloud-ready products across enterprise environments with measurable business impact, clean code quality, and maintainable architecture.

### TECH SKILLS

- ❖ **Languages:** C# 8–12, TypeScript 4–5, JavaScript ES6+, SQL, PowerShell, HTML5, CSS3
- ❖ **Backend:** ASP.NET Core 3.1–8, .NET Framework 4.6–4.8, Web API, Entity Framework Core 3–8, LINQ, SignalR, REST APIs, GraphQL
- ❖ **Frontend:** React 16–18, Redux Toolkit, React Query, TypeScript, Material UI, Bootstrap, Jest, React Testing Library
- ❖ **Cloud & DevOps:** Azure App Service, Azure Functions, Azure DevOps, AWS ECS, AWS Lambda, S3, Docker, Kubernetes, GitHub Actions, Jenkins
- ❖ **Databases:** SQL Server 2016–2022, PostgreSQL 12–15, MongoDB 4–6, Redis, Elasticsearch
- ❖ **Architecture:** Microservices, Clean Architecture, Domain-Driven Design, CQRS, Event-Driven Systems, OAuth2, OpenID Connect
- ❖ **Testing & Quality:** xUnit, NUnit, Moq, Jest, Cypress, SonarQube, Unit Testing, Integration Testing, Agile/Scrum

### PROFESSIONAL EXPERIENCE

#### Staff Software Engineer

##### Relatio

Jun 2022 – Apr 2026

Las Vegas, NV

- ❖ Led development of enterprise SaaS modules using **ASP.NET Core 6–8**, **React 18**, and **TypeScript 5**, improving workflow completion speed by **23%** through reusable UI patterns and optimized API contracts.
- ❖ Designed cloud-ready **microservices** with clean architecture, async messaging, and SQL Server-backed domain boundaries, reducing cross-service dependency issues by **18%** across release cycles.
- ❖ Migrated legacy .NET Core services into **.NET 8**, replacing outdated middleware, dependency injection patterns, and serialization logic while keeping production behavior stable during phased rollout.
- ❖ Resolved recurring memory pressure issues in background workers by profiling async operations, fixing improper scoped service usage, and reducing worker restart incidents by **21%**.
- ❖ Implemented new React dashboard features with **Redux Toolkit**, React Query, role-based access, and reusable data grids to support operational reporting for enterprise users.

- ❖ Improved API response times by **27%** by optimizing Entity Framework Core queries, removing unnecessary eager loading, and introducing Redis caching for high-read endpoints.
- ❖ Strengthened authentication and authorization using **OAuth2**, OpenID Connect, JWT validation, and policy-based authorization across admin and customer-facing applications.
- ❖ Built CI/CD pipelines with GitHub Actions, Docker, Kubernetes manifests, automated tests, and environment approvals to reduce manual deployment steps by **24%**.
- ❖ Mentored mid-level engineers through design reviews, pair programming, and production-readiness checklists focused on **clean code**, test coverage, and service reliability.
- ❖ Fixed React rendering bugs caused by stale query state, improper dependency arrays, and uncontrolled form behavior, improving frontend stability during complex user workflows.
- ❖ Improved production observability with structured logging, correlation IDs, health checks, Azure Application Insights, and alert rules for faster root-cause analysis.
- ❖ Collaborated with product, QA, DevOps, and security teams to translate business requirements into reliable technical delivery plans without over-engineering system complexity.

---

## Senior Software Engineer

### AppFolio

*Feb 2018 – May 2022*

Goleta, CA

- ❖ Developed property technology features using **ASP.NET Core 3.1–6**, **React 16–17**, TypeScript, SQL Server, and AWS services for high-volume leasing and payment workflows.
- ❖ Modernized older MVC and Web API components into service-oriented **.NET Core** modules, improving maintainability and reducing regression defects by **19%**.
- ❖ Built reusable React components for forms, dashboards, search filters, and account workflows, reducing duplicate frontend implementation effort by **22%** across teams.
- ❖ Solved SQL timeout issues by analyzing execution plans, adding targeted indexes, refactoring stored procedures, and improving API reliability during peak traffic.
- ❖ Implemented secure payment-related backend flows with validation, audit logging, retry handling, and role-based access to protect sensitive customer operations.
- ❖ Migrated several legacy JavaScript screens into **React with TypeScript**, improving state management, testability, and accessibility for customer-facing workflows.
- ❖ Created integration tests with xUnit, Moq, and test containers to catch contract-breaking changes before deployment and improve release confidence.
- ❖ Improved AWS-based deployment reliability by refining Docker images, ECS task configuration, environment variables, and rollback procedures for service updates.
- ❖ Reduced production support escalations by **17%** by fixing recurring null-reference errors, race conditions, and inconsistent validation between frontend and backend layers.
- ❖ Participated in architecture reviews, sprint planning, code reviews, and incident retrospectives while helping teams balance delivery speed with long-term code quality.

---

## Software Engineer

### Blackbaud

*Jul 2016 – Jan 2018*

Charleston, SC

- ❖ Built nonprofit CRM and fundraising features using **C#**, **ASP.NET MVC 5**, Web API 2, React 15–16, SQL Server, and JavaScript for donor management workflows.

- ❖ Maintained and enhanced **.NET Framework 4.6–4.7** applications while gradually introducing modern Web API patterns and component-based frontend development.
- ❖ Fixed data synchronization bugs between CRM modules by improving transaction boundaries, validation rules, and retry handling for failed background jobs.
- ❖ Improved report generation performance by **16%** by rewriting inefficient SQL queries, optimizing joins, and reducing unnecessary server-side data transformations.
- ❖ Implemented React-based UI enhancements for donation forms, contact search, and administrative screens while preserving compatibility with existing MVC pages.
- ❖ Resolved authentication-related production issues by strengthening session handling, permission checks, and error logging around sensitive user actions.
- ❖ Built unit tests and integration tests for service-layer logic, helping reduce repeated defects during fundraising campaign releases.
- ❖ Worked closely with QA and product owners to reproduce customer-reported bugs, isolate root causes, and deliver safe fixes under release deadlines.
- ❖ Contributed to code reviews and documentation that improved onboarding for engineers working across legacy .NET and newer frontend modules.

---

## Software Developer

### Granicus

*May 2014 – Jun 2016*

Dallas, TX

- ❖ Developed civic technology applications using **C#, ASP.NET MVC 4–5**, Web API, SQL Server, JavaScript, jQuery, Bootstrap, and early React components.
- ❖ Implemented public-sector workflow features for agenda management, document publishing, notifications, and administrative review processes across web platforms.
- ❖ Improved page-load performance by **14%** by reducing redundant SQL calls, compressing assets, and simplifying controller logic in MVC applications.
- ❖ Fixed production bugs related to document upload failures, timezone inconsistencies, and permission mismatches by improving validation and error handling.
- ❖ Built REST API endpoints for internal integrations, adding request validation, structured responses, and logging to support more reliable partner communication.
- ❖ Supported migration of older Web Forms-style patterns into **ASP.NET MVC**, helping the team standardize routing, controllers, views, and service-layer code.
- ❖ Added automated test coverage for critical business rules and helped reduce manual QA effort for recurring release validation tasks.
- ❖ Collaborated with senior engineers on refactoring tightly coupled modules into cleaner service classes that were easier to test and maintain.

---

## Junior Software Developer

### Vology

*Jun 2013 – Apr 2014*

Gainesville, FL

- ❖ Built internal business applications using **C#, ASP.NET MVC 4**, SQL Server, JavaScript, jQuery, HTML, CSS, and Bootstrap for managed-services operations.
- ❖ Fixed application bugs involving form validation, broken SQL filters, inconsistent UI state, and missing exception handling across support workflows.

- ❖ Improved internal reporting accuracy by **12%** by correcting data mapping logic, validating stored procedure outputs, and reviewing edge cases with business users.
- ❖ Created basic REST-style endpoints and MVC screens for ticket tracking, asset records, and service request workflows under senior engineer guidance.
- ❖ Supported production releases by testing deployments, reviewing logs, documenting defects, and learning maintainable coding practices in an Agile team environment.

## **EDUCATION**

---

### **Arkansas State University**

Bachelor's degree, Computer Science (Sep 2009 – May 2013)